

UT7 - Optimización de código y documentación

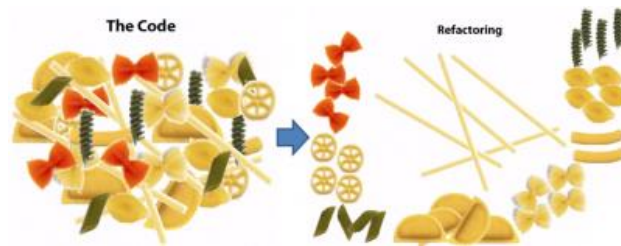
1	Optimización de código. Refactorización.....	2
1.1	Concepto.....	2
1.2	Limitaciones.....	2
1.3	Convenciones de escritura en Java (Code Style)	3
1.4	Buenas prácticas (Best Practices)	5
1.5	Code Smell y métricas de calidad.....	6
1.6	Principios de un buen diseño.....	6
1.7	Patrones de refactorización más usuales.....	6
1.7.1	Herramientas de Eclipse.....	7
1.7.2	Otras Herramientas de ayuda a la refactorización.	8
1.8	Pruebas en la refactorización	8
2	Documentación	9
2.1	Documentación interna	9
2.2	Documentación externa.....	9
2.3	Documentación interna. Uso de comentarios.....	11
2.3.1	Javadoc.....	12
2.3.2	Archivos module-info.java	12
2.3.3	Generación automática de documentación interna Javadoc	13

1 Optimización de código. Refactorización.

1.1 Concepto.

El término refactorizar dentro del campo de la Ingeniería del Software hace referencia a la modificación para la **mejora u optimización del código** sin cambiar su funcionamiento. Se emplea para crear un código más claro y sencillo, facilitando la posterior **lectura, revisión y mantenimiento** (evolutivo, correctivo...) de un programa.

Refactorizar código consiste en crear un código más limpio y eficiente. Se podría entender como el mantenimiento del código, para **facilitar su comprensión**, pero sin añadir ni eliminar funcionalidades



La refactorización debe ser un paso aislado, para evitar introducir errores de código al reescribir o modificar algunas partes del mismo. Si después de refactorizar hemos alterado el funcionamiento del código, hemos cometido errores al refactorizar.

Se refactoriza para:

- **Limpieza del código**, mejorando la comprensión, lectura y claridad.
- **Facilitar el mantenimiento del código**, introducir modificaciones, solucionar bugs, etc
- **Mejorar el rendimiento y eficiencia.**
- **Eliminar el código “muerto” y duplicado.**

1.2 Limitaciones.

Aunque la refactorización puede mejorar significativamente la calidad del código, también tiene limitaciones. Algunas de estas limitaciones incluyen:

- **Riesgo de introducir errores:** Existe el riesgo de introducir nuevos errores durante el proceso de refactorización, especialmente si no se realizan pruebas adecuadas.
- **Tiempo y recursos:** La refactorización puede ser un proceso intensivo en tiempo y recursos, especialmente para grandes bases de código o sistemas complejos.
- **Necesidad de comprensión del código:** Para realizar una refactorización efectiva, los desarrolladores deben comprender completamente el código existente y sus implicaciones.
- **Impacto en el flujo de trabajo:** La refactorización puede afectar el flujo de trabajo del equipo, especialmente si se realiza en un código que está siendo utilizado activamente en un proyecto en curso.

1.3 Convenciones de escritura en Java (Code Style)

Las **convenciones de código o code style** existen debido a que la mayoría del coste del código de un programa se usa en su mantenimiento, (casi ningún programa se mantiene toda su vida con el código original), y mejoran la lectura del código permitiendo entender código nuevo mucho más rápido y a fondo. En la web de *Sun Microsystems* se recogen dichas convenciones en una guía en [ingles](#), o una traducción al [castellano](#) cortesía de *javaHispano*. Por su parte Google también ha creado recientemente una [Guía de estilo](#) para Java.

Para que las convenciones funcionen, cada programador debe tratar de ser lo más fiel posible a estas.

Ficheros

Todos los ficheros fuente de java son ficheros de texto plano cuyo nombre termina con la extensión `.java`. Dentro de cada fichero `.java` tenemos 4 partes en el siguiente orden:

1. Posibles comentarios sobre la clase (autor, fecha, licencias, etc)
2. Sentencia *package*. Toda clase debe estar en un paquete.
3. Sentencias *import*. Importar cada clase en una línea separada.
4. Definición de una única clase o interface por fichero **cuyo nombre debe coincidir con el nombre del fichero `.java`**.

```
/* Correcto */           /* Incorrecto */
import java.awt.Frame;    import java.awt.*;
import java.awt.Graphics;
```

Posteriormente dentro de la definición de la clase, aplicamos el siguiente orden:

1. Sentencia *class* o *interface*
2. Variables de clase (static)
3. Constantes
4. Variables de instancia (Atributos de la clase)
5. Constructores (Si hay sobrecarga deben ir seguidos)
6. Métodos (Si hay sobrecarga deben ir seguidos)

Declaraciones de variables

- Una sola declaración por línea

```
int edad;
int cantidad;
```

- Las variables locales se deben inicializar en el momento de declararlas o justo después. Se declaran justo antes de su uso, para reducir su ámbito.
- Las variables de instancia o miembros de clase se declaran al comienzo de la definición de la clase.
- Los arrays se pueden inicializar en bloque:

```
int[] array =
{
    0, 1, 2, 3
};
// ó
int[] array = { 0, 1, 2, 3 };
```

Nombres de identificadores

Para los identificadores podemos usar las letras anglosajonas y números de la tabla ASCII. No se recomienda usar caracteres con tilde ni la (ñ). Las barras bajas o guiones tampoco se recomiendan. **Los nombres de los identificadores deben ser siempre lo más descriptivos posible, ya sea variable, método o clase.**

- Nombre de **Package**: siempre en minúsculas.
- Nombre de las **clases o interfaces**: *UpperCamelCase*.
- Nombre de los **métodos**: *lowerCamelCase*. Suelen ser verbos o frases.
- Nombres de **constantes**: *CONSTANT_CASE*. Todo el mayúsculas, separando con barra baja.
- **Variables locales, atributos de la clase, nombres de parámetros**: *lowerCamelCase*.

Magic Numbers

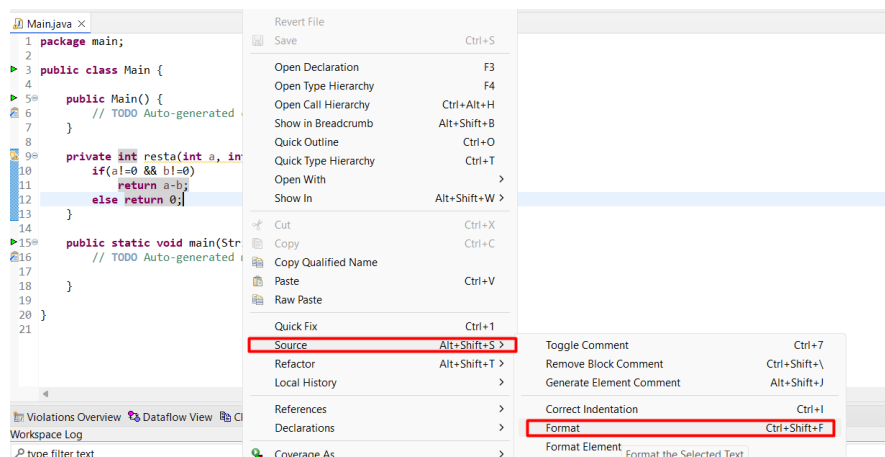
Se conoce bajo este nombre a cualquier valor literal ("texto" o numérico) empleado en el código sin ninguna explicación. Se deben sustituir siempre que se pueda por una constante que identifique su finalidad.

```
//incorrecto
int precioConIva = precioBase + (0.21 * precioBase);

//correcto
//Se define en la clase
final static double IVA = 0.21;
//Se utiliza en un método
int precioConIva = precioBase + (IVA * precioBase);
```

Recomendaciones en la estructura del código (Format)

- En las sentencias de control de flujo (if, else, for, do-while, try-catch-finally) se deben **utilizar siempre llaves { }**. Se alinean las llaves {} al inicio de línea.
- **Una sola instrucción por línea.**
- Las **líneas de código no deben superar los 100 caracteres**. Se deben separar antes de algún operador.
- **Si la declaración de un método es demasiado larga**, o una expresión aritmética es demasiado larga, se debe **separar en distintas líneas**.
- Si una **operación aritmética o lógica** es compleja se deben **usar paréntesis para facilitar su legibilidad**.
- Los **espacios en blanco mejoran la legibilidad**. Se deben colocar entre operadores, después de los puntos y coma de los bucles for, después de los operadores de asignación, etc.
- **Atajo en Eclipse: Ctrl + Shift + F (format), formatea la clase con esta estructura:**



```
if (final < indice) {
    filaInicial = indice - numeroFilas;
} else if (indice < filaInicial) {
    filaInicial = indice;
}
```

```

public void ejecutarAccion(
    TipoParametro parametro1, TipoParametro parametro2, TipoParametro parametro3){
    ...
}

if ((condicion1 && condicion2)
    || (condicion3 && condicion4)
    || !(condicion5 && condicion6)) {
    llamarMetodo();
}

longName1 = longName2 * (longName3 + longName4 - longName5)
+ (4 * longname6);           //Siempre con el operador al principio de línea

```

```

cantidadTotal = cantidadInicial + cantidadFinal;

for(int i = 0; i < cantidadTotal; i++){
    ...
}

public String getItem(int fila, int columna) {
    ...
}

getItem(cantidadInicial, cantidadFinal);

```

1.4 Buenas prácticas (Best Practices)

- **Manejo de Strings:** Los Strings son objetos, por lo que crearlos es costoso.
 - Es mucho **más rápido instanciarlos con una asignación, que** con el operador **new**.
 - **Evitar concatenar String con el operador '+'** también genera mucha carga, ya que crea un nuevo String en memoria (Los objetos String son inmutables). Se debe tratar de evitar siempre las concatenaciones (+) dentro de un bucle, o usar otras clases en ese caso (p.e. StringBuilder)

```

//instanciación lenta
String lenta = new String("objeto string");

//instanciación rápida
String rapida = "objeto string";

```

- **Tipos primitivos mejor que clases *wrapper* (envoltorio)** : Las clases wrapper al ser objetos, proveen de métodos para trabajar mejor con ellas, pero al igual que los Strings, son más lentos que los tipos primitivos.
- **Comparación de objetos:** Recordar que tanto los *Strings* como las tipos *Wrapper* son objetos y sus variables solo contienen sus referencias (direcciones). **Los objetos no se comparan con ==.**
- **Evitar la creación innecesaria de objetos.** Como se ha dicho, generan mucha carga.

```

int x = 10;
int y = 10;

Integer x1 = new Integer(10);
Integer y1 = new Integer(10);

String x2="hola";
String y2 = new String("hola");

System.out.println(x == y);    //TRUE
System.out.println(x1 == y1);  //FALSE, ya que son 2 objetos distintos
System.out.println(x2 == y2);  //FALSE, ya que son 2 objetos distintos

```

- **Visibilidad de atributos:** Los **campos de una clase** 'estándar' no deben declararse **nunca como public**, ni mucho menos no indicarle un modificador de visibilidad. Se debe **usar sus *setters* y *getters*** para su acceso.
- **Usar las variables para un único propósito.** A veces sentimos la tentación de reutilizar una variable, pero complica la legibilidad.

```

...
int resultadoTotal = resultadoInicial - resultadoFinal;
...

```

- **Bucle for. Optar por el for** siempre que se pueda (**frente a while, do-while**). Las ventajas son que reúne todo el control del bucle en la misma línea (inicio, fin, e incremento), y la variable de control ('i') no es accesible desde fuera de él. Si se necesita modificar su variable de control, usar otro bucle.

- **Constantes:** Cualquier valor literal debe ser definido como constante, excepto 1, -1, 0 ó 2 que son usados por el bucle for.
- **Switch:** Siempre debe llevar un *break* después de cada caso, y también el caso *default*.

1.5 Code Smell y métricas de calidad

Se conoce como **Bad Smell o Code Smell** (mal olor) a algunos indicadores o síntomas del código que posiblemente ocultan un problema que dificulta el posterior mantenimiento del mismo y aumentan el riesgo de errores futuros. Algunos de estos síntomas son:

- **Código duplicado** (Duplicated code). Si se detectan bloques de código iguales o muy parecidos en distintas partes del programa, se debe extraer creando un método para unificarlo.
- **Métodos muy largos** (Long Method). Un método largo probablemente está realizando distintas tareas, que se podrían dividir en otros métodos. Las funciones deben ser las más pequeñas posibles (3 líneas mejor que 15). Cuanto más corto es un método, más fácil es reutilizarlo. Un método debe hacer solo una cosa, hacerla bien, y que sea la única que haga.
- **Clases muy grandes** (Large class). Una clase debe tener solo una finalidad. Si una clase se usa para distintos problemas tendremos clases con demasiados métodos, atributos e incluso instancias. Las clases deben el menor número de responsabilidades y que esté bien delimitado (alta cohesión).
- **Lista de parámetros extensa** (Long parameter list). Las funciones (o métodos en Java) deben tener el mínimo número de parámetros mínimo necesario. Si un método requiere muchos parámetros puede que sea necesario crear una clase con esa cantidad de datos y pasarle un objeto de la clase como parámetro.

Existen métricas que miden la calidad del código. Algunos ejemplos son:

- **CBO** (Coupling Between Objects): Mide el acoplamiento, el número de clases externas con las que una clase está acoplada (usa atributos o métodos de otras clases). Un **CBO bajo** es ideal, indica poca dependencia de otras clases.
- **WMC** (Weighted Methods per Class): mide del número de métodos y su complejidad, para calcular la complejidad de la clase. Un valor **WMC bajo** es lo ideal.
- **LCOM** (Lack of Cohesion of Methods): Mide la falta de cohesión entre los métodos de una clase a través de los atributos compartidos que utilizan. Un **valor bajo LCOM** indica que existe una buena cohesión.

1.6 Principios de un buen diseño

En el paradigma de la programación orientada a objetos son pilares clave de un buen diseño los siguientes principios básicos:

- **Bajo acoplamiento:** Las clases del sistema deben depender lo menos posible entre sí.
- **Principio de responsabilidad única:** las clases y métodos deben tener una única responsabilidad.
- **Alta cohesión:** Los métodos y atributos dentro de un módulo o clase deben estar estrechamente relacionados y trabajar juntos para cumplir una única responsabilidad.

1.7 Patrones de refactorización más usuales

Existen numerosos patrones de refactorización que los desarrolladores utilizan para mejorar el código. Algunos de los más comunes incluyen:

1. **Extracción de métodos:** Dividir un método largo en métodos más pequeños y específicos para mejorar la legibilidad y la reutilización del código.
2. **Renombrado de variables y métodos:** Utilizar nombres más descriptivos y significativos para mejorar la comprensión del código.
3. **Eliminación de código duplicado:** Identificar y eliminar código duplicado para mejorar la mantenibilidad y evitar inconsistencias.
4. **Reorganización de métodos:** Reordenar métodos y funciones para mejorar la cohesión y reducir la complejidad.
5. **Simplificación de expresiones condicionales:** Simplificar expresiones condicionales complejas para mejorar la claridad y la legibilidad del código.

1.7.1 Herramientas de Eclipse

Eclipse tiene distintos métodos de refactorización. Dependiendo de sobre qué mostremos el menú de refactorización, nos ofrecerá unas u otras opciones. Para refactorizar pulsaremos click derecho sobre el nombre del elemento deseado, y desplegaremos la opción **Refactor en Eclipse** del menu contextual.

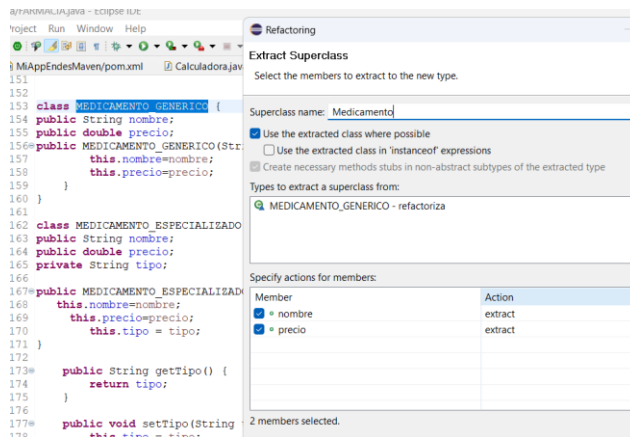
CLASE	MÉTODO	ATRIBUTO
Rename... Alt+Shift+R Move... Alt+Shift+V Extract Interface... Extract Superclass... Use Supertype Where Possible... Pull Up... Push Down... Extract Class... Generalize Declared Type... Infer Generic Type Arguments...	Rename... Alt+Shift+R Move... Alt+Shift+V Change Method Signature... Alt+Shift+C Inline... Alt+Shift+I Extract Interface... Extract Superclass... Use Supertype Where Possible... Pull Up... Push Down... Extract Class... Introduce Parameter Object... Introduce Indirection... Infer Generic Type Arguments...	Rename... Alt+Shift+R Move... Alt+Shift+V Extract Interface... Extract Superclass... Use Supertype Where Possible... Pull Up... Push Down... Extract Class... Encapsulate Field... Generalize Declared Type... Infer Generic Type Arguments...

Los **métodos de refactorización**, también llamados **patrones de refactorización**, nos permiten plantear casos y previsualizar las posibles soluciones que se nos ofrecen. A continuación se muestran algunos de los métodos más comunes:

- **Rename:** Es la opción empleada para cambiar el identificador a cualquier elemento (nombre de variable, clase, método, paquete, directorio, etc). Cuando lo aplicamos, se cambian todas las veces que aparece dicho identificador.
- **Move:** Mueve una clase (archivo .java) de un paquete a otro y se cambian todas las referencias.
- **Extract Constant:** Convierte un número o cadena literal en una constante.
- **Extract Local Variable:** Convierte un número o cadena literal en una variable de ámbito local.
- **Convert Local Variable to Field:** Convierte una variable local en un atributo privado de la clase.
- **Extract Method:** Convierte un bloque de código en un método, a partir de un bloque cerrado por llaves { }. Eclipse ajusta los parámetros y el retorno del método. Es muy útil cuando detectamos métodos muy largos, o en bloques de código que se repiten.
- **Change Method Signature:** Permite cambiar el nombre del método y los parámetros que recibe. Se actualizarán todas las dependencias y llamadas al método dentro del proyecto actual.
- **Inline:** Nos permite ajustar una referencia a una variable o método en una sola línea de código.

```
File fichero = new File("datos.dat");  
PrintWriter escritor;  
escritor = new PrintWriter(fichero);  
PrintWriter escritor = new PrintWriter(new File("datos.dat"));
```

- **Extract Interface:** nos permite seleccionar los métodos de una clase para crear una Interface. Define los métodos (Nombre, parámetros y tipo de retorno).
- **Extract Superclass:** Permite crear una superclase (clase padre) con los métodos y atributos comunes que seleccionemos de una clase concreta.



1.7.2 Otras Herramientas de ayuda a la refactorización.

Control de versiones (Git, SVN): Las herramientas de control de versiones, como Git, permiten realizar cambios de forma segura y revertirlos si es necesario durante el proceso de refactorización

Analizadores de Código estático

- **Linters:** Herramientas de análisis estático que identifican problemas potenciales en el código, como código duplicado, variables no utilizadas o prácticas de codificación inconsistentes.
- **Code metrics tools:** Herramientas que proporcionan métricas sobre la calidad del código, como la complejidad ciclomática, la duplicación de código y la profundidad de la herencia, lo que puede ayudar a identificar áreas que podrían beneficiarse de la refactorización.
- **Ejemplos:** Eclipse PMD plugin, IntelliJ TreeMetrics

1.8 Pruebas en la refactorización

La refactorización y las pruebas son procesos complementarios en el desarrollo de software.

Es importante realizar **pruebas exhaustivas antes y después de la refactorización** para garantizar que el comportamiento del código no se vea alterado y no se hayan introducido nuevos errores en el código.

1. **Pruebas unitarias:** Verifican el funcionamiento correcto de las unidades individuales (módulos, librerías, métodos, funciones o clases).
2. **Pruebas de integración:** Evalúan cómo los diferentes módulos interactúan entre sí y si siguen interactuando correctamente.
3. **Pruebas funcionales:** Comprueban si el sistema sigue cumpliendo con los requisitos funcionales especificados.
4. **Pruebas de regresión:** Garantizan que los cambios realizados no han roto funcionalidades que ya funcionaban.
5. **Pruebas de aceptación:** determinan si los cambios son aceptables desde el punto de vista del usuario final.
6. **Pruebas de rendimiento:** verifican si la refactorización ha afectado al rendimiento general del sistema
7. **Pruebas de seguridad:** Evalúan si la refactorización ha introducido vulnerabilidades.

Estas pruebas se especifican en el **plan de pruebas**, documento detallado que describe el enfoque, las actividades y los recursos necesarios para llevar a cabo el proceso de pruebas de software. Sirve como guía para los equipos de prueba y desarrollo.

2 Documentación

La documentación es fundamental para facilitar el entendimiento, desarrollo y mantenimiento de los programas. Existen 2 grandes grupos de documentación, interna y externa.

2.1 Documentación interna

Integrada dentro del código fuente del programa. Enfocada a facilitar a los programadores el mantenimiento del código.

A parte de utilizar convenciones de escritura y buenas prácticas de programación se recomienda el uso de comentarios en el código a medida que se escribe.

2.2 Documentación externa

El enfoque y la cantidad de documentación externa que se genere pueden variar significativamente dependiendo de la metodología de desarrollo de software que se utilice.

La documentación es un elemento vivo que evoluciona y se actualiza continuamente durante el ciclo de vida del proyecto. No es algo estático o cerrado, sino un elemento dinámico que se adapta a los cambios en los requisitos, la implementación, el mantenimiento, las necesidades, etc

Metodologías tradicionales (cascada, espiral)

- *Enfoque:* Documentación extensa y detallada.
- *Características:*
 - Se crea mucha documentación externa durante todas las fases del proyecto (requisitos, diseño, implementación, pruebas y mantenimiento).
 - La documentación suele incluir especificaciones detalladas, diagramas de diseño (UML), casos de prueba formales, etc.
- *Ejemplo:* Documentación de requisitos, diseño del sistema, manuales de usuario, plan de pruebas.

Metodologías ágiles (Scrum, Kanban, XP)

- *Enfoque:* "Documentación mínima necesaria" 
- *Características:*
 - Se prioriza la entrega de software funcional por encima de la documentación exhaustiva.
 - En lugar de documentos extensos, se utiliza la comunicación continua (reuniones de equipo, sprints, retrospectivas).
 - Se puede usar documentación ligera, como historias de usuario (descripciones de la funcionalidad desde el punto de vista del usuario), casos de prueba esenciales o diagramas simples pero efectivos.
- La documentación suele estar enfocada en lo esencial: manuales de usuario básicos y documentación técnica mínima para mantenimiento.

Metodologías híbridas

- *Enfoque:* Balance entre agilidad y documentación formal.
- *Características:*
 - Se documenta lo necesario para asegurar la mantenibilidad y escalabilidad del sistema.
 - Se utilizan herramientas automatizadas para generar documentación técnica (por ejemplo, Javadoc).
 - Puede haber documentación ligera y/o detallada y extensa en función de la complejidad y tamaño del proyecto y las necesidades.

Tipos de documentación externa (Ejemplos)

Esta documentación abarca toda la documentación generada durante el ciclo de vida del software y de la gestión del proyecto.

- *Documentación de gestión del proyecto*
 - **Acta de constitución del proyecto:** descripción inicial del proyecto, propósito y objetivos, identificación de stakeholders o interesados del proyecto.
 - **Análisis de viabilidad técnica y económica:** análisis costo-beneficio (ACB), y de retorno de la inversión (ROI)
 - **Planes de gestión** (alcance, riesgos, calidad, comunicación, cronograma, costos, recursos humanos, etc)
 - **Desglose de tareas:** EDT (Estructura de Desglose del Trabajo) o WBS (Work Breakdown Structure). Divide el proyecto en entregables y paquetes de trabajo, facilitando su planificación, ejecución y control
 - **Pliego de condiciones del proyecto:** enfoque contractual (plazos, condiciones legales, etc).
- *Documentación de requisitos:* Especificación de requisitos funcionales y no funcionales
 - **Descripción del alcance y la funcionalidad** del sistema
 - **Análisis de casos de uso** (diagramas UML de casos de uso)
 - **Requisitos no funcionales**, entorno tecnológico, infraestructura, estándares y normas
- *Documentación técnica:* Especificación de soluciones técnicas, guía para el desarrollo.
 - **Arquitectura del sistema** (diseño de la arquitectura software y hardware)
 - **Tecnologías utilizadas**
 - **Documentación de diseño** (diagramas UML de clases, de paquetes, de despliegue, modelo de datos ER, Relacional, etc)
- *Documentación de pruebas*
 - **Plan de pruebas:** enfoque, recursos destinados a la prueba, identificación de los elementos a probar, entre otros.
 - **Diseño de las pruebas:** descripción de fallo y validación de la prueba, selección de conjuntos de datos para las pruebas, entre otros.
 - **Historial de pruebas realizadas y resultados**
- *Documentación de cliente*
 - **Manual de explotación:** instalación y configuración del sistema.
 - **Manual de usuario:** descripción del uso del sistema desde el punto de vista de los diferentes roles de usuario (administrador, cliente, operario, empleado, etc)

2.3 Documentación interna. Uso de comentarios

Los comentarios en Java son una forma de documentar el código para que otros desarrolladores (y tú mismo en el futuro) puedan entender su funcionalidad sin necesidad de analizar el código fuente en detalle. Existen dos tipos principales de comentarios en Java: comentarios de una línea y comentarios de varias líneas.

Comentarios de una línea:

Se utilizan para agregar breves explicaciones sobre una línea de código específica.

```
int x = 5; // Inicializa la variable x con el valor 5
```

Comentarios de varias líneas:

Se utilizan para explicar bloques de código más extensos o para agregar documentación detallada sobre clases, métodos, variables, etc.

```
/*
 * Este es un comentario de varias líneas
 * Puede abarcar múltiples líneas de código
 * y se usa comúnmente para explicar la funcionalidad
 * de un fragmento de código.
 */
```

Comentarios de documentación Javadoc™:

En cuanto a Javadoc, es una herramienta que viene incluida en el JDK de Java y permite generar documentación a partir de los comentarios del código fuente en un formato específico. Los comentarios Javadoc comienzan con `/**` y pueden incluir etiquetas especiales como `@param`, `@return`, `@throws`, etc., que ayudan a estructurar la documentación generada.

Aquí tienes un ejemplo de cómo se vería un comentario Javadoc para documentar un método:

```
/**
 * Calcula el área de un rectángulo.
 *
 * @param largo La longitud del rectángulo.
 * @param ancho La anchura del rectángulo.
 * @return El área del rectángulo.
 * @throws IllegalArgumentException Si el largo o el ancho son negativos.
 */
public double calcularAreaRectangulo(double largo, double ancho) {
    if (largo < 0 || ancho < 0) {
        throw new IllegalArgumentException("El largo y el ancho deben ser valores positivos.");
    }
    return largo * ancho;
}
```

Etiquetas comunes en Javadoc:

- `@param`: Describe los parámetros del método.
- `@return`: Describe el valor de retorno del método.
- `@throws` o `@exception`: Indica las excepciones que puede lanzar el método.
- `@author`: Indica el autor del código.
- `@version`: Describe la versión del programa.

2.3.1 Javadoc

El programa Javadoc se encuentra en el directorio bin de la JDK de Java.

Sintaxis de comando Javadoc:

```
javadoc -d [directoriosalida] [archivo.java]
```

Ejemplo:

```
javadoc -d docs MiClase.java
```

- `-d docs`: Especifica que la documentación generada se guardará en una carpeta llamada `docs` (puedes cambiarla por otro nombre si quieres).
- `MiClase.java`: Es el archivo fuente de la clase.

Opciones del comando Javadoc:

- `-author`: Incluye información del autor en la documentación.
- `-version`: Incluye la versión del código.
- `-sourcepath`: Permite especificar el directorio raíz donde se encuentran los archivos fuente.

```
javadoc -d docs -author -version -sourcepath ./src MiClase.java
```

Esto generará documentación detallada con el autor y la versión del código.

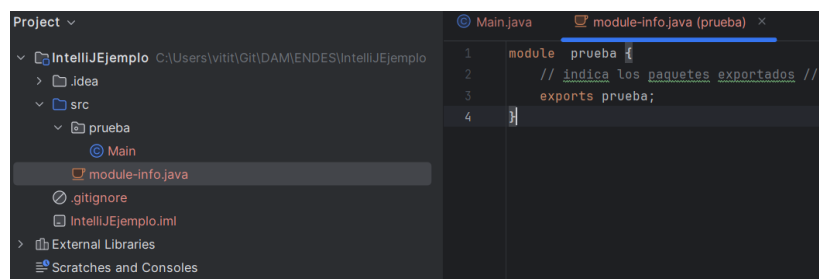
2.3.2 Archivos module-info.java

module-info.java es un archivo especial introducido en **Java 9** que define y documenta un **módulo**, una unidad de encapsulación que organiza el código en proyectos modulares. Sirve para **declarar dependencias** y **controlar qué partes del código pueden ser usadas por otros módulos**.

Su información aparecerá en la documentación Javadoc.

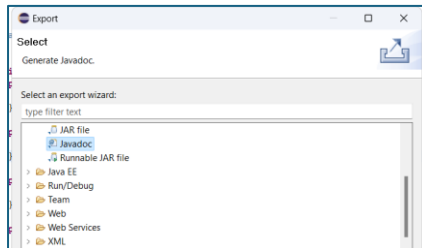
Un archivo module-info.java debe contener una **declaración de módulo** válida con al menos una de las siguientes directivas:

1. **module nombre {}** → Declara el módulo.
2. **requires** → Especifica módulos que se necesitan.
3. **exports** → Permite que otros módulos usen un paquete.

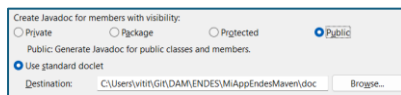


2.3.3 Generación automática de documentación interna Javadoc

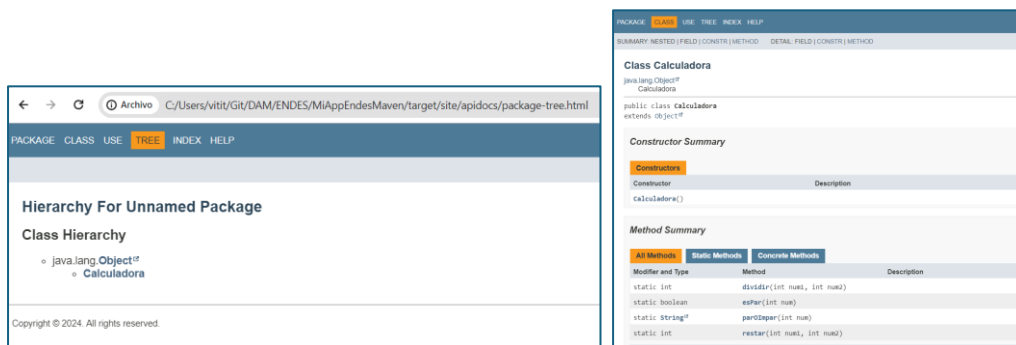
En Eclipse podemos generar automáticamente todo el Javadoc del proyecto desde, botón derecho sobre proyecto >> **Export...**



Escogemos el proyecto, nivel de visibilidad que queremos que aparezca en la documentación y directorio de generación:



Se generará documentación de los comentarios Javadoc en formato **html** con hipervínculos para navegar fácilmente por ella.



Desde IntelliJ

<https://www.jetbrains.com/help/idea/javadoocs.html#generate-javadoc>

Tools > Generate Javadoc...

